

Full length article

Design of an IoT-PLC: A containerized programmable logical controller for the industry 4.0[☆]Jacob Mellado, Felipe Núñez^{*}

Department of Electrical Engineering, Pontificia Universidad Católica de Chile, Av. Vicuña Mackenna 4860, Santiago, 7820436, Chile

ARTICLE INFO

Keywords:

Industrial Cyber-Physical Systems
Industry 4.0
Industrial control
Internet of Things
Fog computing

ABSTRACT

The Programmable-logical-controller (PLC) has been the key building block of industrial control systems throughout the whole automation revolution, where its role has been mainly to command low-level regulatory feedback control loops. Despite the great recent advances in automation technologies, driven by the paradigm of the Industry 4.0 and its hyper-connected ecosystem, the PLC has not seen yet a modernized version targeting the functionalities an Industry 4.0-oriented control system requires. In this work, a device named IoT-PLC is designed and prototyped, in an effort to generate a PLC tailored for the Industry 4.0 revolution. The proposed IoT-PLC operates as a containerized piece of equipment, with each functionality packaged within a separate container. The IoT-PLC has regulatory control capabilities, fog-computing functionalities as filtering and field data storage, and multiple wireless interfaces managed independently. Moreover, it uses a virtual device model that works as an abstraction method to represent real entities, so that IoT-PLC applications can interact transparently and with straightforward compatibility with upper cloud layers. Live migration functionalities add flexibility by allowing a loop reconfiguration without restarting the controller. Laboratory experiments illustrate the effectiveness and potential of the proposed device.

1. Introduction

In the current industrial realm, the term Industry 4.0 has gained increasing attention. Industry 4.0 refers to the integration of a set of digital technologies in intelligent production and advanced information systems, looking to achieve a higher level of operational efficiency and productivity [1]. From a systems and control perspective, this integration gives rise to the concept of Industrial Cyber-Physical Systems (ICPSs) [2], which can be regarded as the next generation industrial control systems (ICSs) and promise to revolutionize large-scale industrial facilities by enabling the acquisition and highly efficient analysis of large amounts of data [3,4]. Under this paradigm, the Industry 4.0 requires units involved in production processes to be smarter, smaller and modular according to the tasks in which they are involved [5].

The Industry 4.0 imposes several design principles that must be considered in the automation or digitalization of production processes [6], namely: (i) integration, particularly of production processes into the value chain, enabling smart factories to perform an end-to-end management of the entire manufacturing process from supply chain to services and lifecycle management; (ii) interoperability, which refers to the capability of all system components to connect, communicate, and interact transparently using standardized communications and data

models; (iii) decentralization, which enables the different sub-systems to make autonomous local decisions aiming at achieving a ultimate organizational goal; (iv) real time performance, which requires that production processes collect, communicate, and consolidate data at the proper rates so strategic decisions can be taken timely; and (v) modularity, which ensures that individual components can be replaced, expanded, or improved with the minimum disruption to production processes, hence allowing smart factories to easily adapt to changing physical environments and production requirements.

Transitioning from the current setup of an industrial production facility, governed by classical ICSs, to an Industry 4.0 setting requires a progressive transformation, involving an adaptation of the devices and architectures used in ICSs to fit the ICPS paradigm in order to achieve both vertical and horizontal integration of devices at all levels [7].

Among the devices that need to undergo a transformation is the programmable logical controller (PLC). The PLC has been the key building block of ICSs throughout the whole automation revolution, where its role has been mainly to command low-level time-sensitive regulatory feedback control loops, providing a first control layer for the processes. Since sensors and actuators are commodity hardware available for

[☆] This work was partially funded by ANID, Chile under grant ANID PIA ACT192013.

^{*} Corresponding author.

E-mail addresses: jemellado@uc.cl (J. Mellado), fenuñez@uc.cl (F. Núñez).

Table 1

Typical requirements in industrial automation and control domains [10]. Reliability refers to how sensitive applications are to the loss of packets.

Application domain	Typical cycle time	Reliability level
Building automation	10 s	Medium
Process automation	100 ms	Medium
Factory automation	1 ms	High
Power system protection	100 μ s	High
Power electronics control	10 μ s	Very high

buying from numerous vendors, the PLC is the first element directly related to the control infrastructure [8].

For decades, the general architecture of the PLC has been invariant [9], where the controller hardware can be considered a typical embedded system and the controller software consists of an operating system and standardized communication stacks, over which the controller firmware generates its domain-specific functionality [9]. The main subsystems of the firmware are the execution engine for control programs, the database for storing relevant runtime data, the operator interaction mechanism and the input/output system. For the upcoming Industry 4.0 era, this architecture should be updated to accommodate new technologies as: wireless instrumentation, cloud-based management systems, and autonomous field elements, while maintaining a reliable and predictable behavior.

Fulfilling tight timeliness constraints, typical in the operational technologies (OT) domain, is a key functionality of a PLC that should be ensured in any new development. Timeliness constraints vary depending on the application domain, with typical cycle times ranging from microseconds to several seconds, as illustrated in Table 1. Furthermore, an Industry 4.0 PLC must also be interoperable and ready to integrate with platforms from the information technologies (IT) domain, where strong timeliness constraints are not present and deterministic communications are not critical. Therefore, a modern PLC is by definition an element facilitating the OT-IT integration, which is a difficult problem itself [7].

To solve the integration problem between the IT and OT domains, while satisfying their requirements, the fog computing paradigm has appeared as an appealing option to deploy control systems, storage and communications closer to the field devices. Using this paradigm, response times and non-deterministic behaviors of the IT layers can be reduced, given that part of the tasks performed in the IT area are moved closer to sensors and actuators [11]. In this context, conceiving a modern PLC as a fog node seems promising.

This work proposes a container-based architecture for an Industry 4.0-oriented PLC inspired in the Internet of Things (IoT) paradigm, called IoT-PLC, that allows flexible deployment of controllers, direct integration with cloud-based management systems, and efficient communications with wireless instrumentation. The proposed IoT-PLC describes each field device as a resource (virtual device) for edge and cloud applications, enabling a smooth, standardized and robust information flow from field to cloud. Containerization allows live migration of applications between IoT-PLCs, enabling hot reconfiguration of control loops, and fine tuning of computing resources assigned to each process inside the IoT-PLC, increasing robustness and predictability.

The rest of this paper is organized as follows: Section 2 describes related work. Section 3 details the proposed architecture for the IoT-PLC. Section 4 presents a prototyping exercise and performance evaluation. Section 5 concludes the paper with remarks and directions for future research.

2. Related work

Ongoing research exists on modern PLC design and prototyping, focusing mainly on enhancing functionalities.

An initial effort in this direction is the work in [12], which proposes the concept of control-as-a-service by using virtualized PLCs inside a

cloud-environment. The authors conclude that the scheme is suitable for soft real-time applications, no details on the capabilities of the virtualized PLC are given.

The idea of virtualized PLCs is pushed further in [13], where by leveraging virtualization and advanced communication technologies PLC functionalities and physical input/output interfaces are decoupled, yielding a real-time virtual machine hosted on a real-time hypervisor, connected to input/output modules on the field using a switched deterministic Ethernet system. The conceptual scheme in [13] focuses on ultra low latency applications and does not target new functionalities as wireless interfaces and high-level connectivity.

A more concrete effort is the work in [14], which proposes a gateway playing the role of a PLC to connect devices to the cloud based on virtual instances and MQTT messages. The gateway describes field devices using the FiWare model, in this way each field device has its characteristics attached to its virtual instance. Although relevant from an information modeling point of view, the device designed in [14] is conceived to, merely, connect devices.

The cloud soft-PLC designed in [15] looks to modernize the PLC by increasing its scalability and making it cost efficient. In this soft-PLC, OPC-UA is used to send sensor data to the cloud where the PLC is running completely as a piece of software. This cloud-controller includes database, cache and tools for human interaction. The advantages of the soft-PLC are its horizontal scalability and task diversification, but the main issue is the latency between field devices and the cloud controller.

The framework proposed in [16] aims to address the large latency of cloud controllers by building an infrastructure for integration of fog-based IoT control systems. The data transmission strategy is designed for real-time control applications and adds fog-to-fog node communication over a simple transmission model, where fog nodes play the role of a PLC. The system includes applications running in fog nodes and is tailored for time-aware processing within the distributed message-oriented middleware. This PLC scheme has as advantages its design as fog node and its horizontal integration; yet, it does not define a device description, which precludes the integration with cloud backends, and applications share resources, which reduces reliability and precludes predictability and real-time performance.

To isolate every application in the fog node a multicontainer and micro-service-based framework for a PLC-like IoT gateway is presented in [17], here the containerized services perform common gateway functions like device discovery, data management and cloud integration. The proposed framework can be adapted to perform real-time control while connectivity with the cloud is available, yet the design does not allow fast and lightweight deployment of control loops with its virtual devices and interfaces.

In [18] is proposed to provide control services at an edge computing gateway using deterministic IP-based industrial communication. This scheme provides different applications and services to field devices such as: virtual control functions, data analytics and visualization. By exploiting deterministic IP-based communications, latency from edge to field when implementing a (wired) Profinet over TCP/IP communication scheme is reduced to 17 μ s for a single-hop scenario, way below the typical requirements of process control which are in the hundreds of milliseconds range [10]. Control performance is not explored.

An architecture for control devices is introduced in [19], referred to as PLC 4.0, that operates as a bridge between industrial automation applications and IT technologies and provides a virtual platform for testing, which runs in parallel to the control procedure. The work in [19] deals with a new architecture for PLCs, which applies both container technologies and hypervisor solutions to ensure dynamic deployment and secure connectivity as well as separation of critical applications, real time Operating Systems (OS) and communication. Furthermore, it provides an abstraction layer that increases the agility of the PLC and enhances the communication Quality of Service (QoS). The proposed architecture allows the control logic to be changed without affecting performance.

The proposed IoT-PLC incorporates new elements with respect to existing efforts, namely: (i) the use of containers to manage control-oriented services in a modular fashion with resources assigned to prioritize control functions and isolate failures; (ii) block reconfiguration without stopping the control process and thus not affecting the performance of the control loop; and (iii) on-the-fly migration of control tasks to a second device, enabling fast deployment of a control loop without having to start all the tasks from scratch. Table 2 presents a summary of the main characteristics of previous efforts and the proposed IoT-PLC.

3. IoT-PLC design

The proposed IoT-PLC is conceived as an Industry 4.0 PLC. It includes a variety of wireless interfaces for field level connectivity with sensors and actuators. Additionally, it contains, internally, a filtering module for pre-processing of data streams, and a lightweight local database for short term trend analysis and display via its human machine interface (HMI). A manager process is in charge of generating control loops using field sensors and actuators and instantiating controllers in the form of containers for low level regulatory control. For high level connectivity, it contains two interfaces, namely, a dedicated interface for cloud connectivity capable of providing an standardized information model for integration with a variety of control systems and a lightweight interface for horizontal connectivity with other IoT-PLCs deployed in the field. To implement the former, OPC-UA is selected, while for the latter CoAP (constrained application protocol) is chosen as the nominal protocol. The CoAP interface is useful when an IoT-PLC needs to form a closed-loop with a wireless sensor or actuator outside its wireless range and uses a second IoT-PLC as relay node. This horizontal connectivity solution potentially reduces the latency with respect to a vertical integration solution where the cloud acts as a bridge.

The proposed PLC scheme complies with the required functionalities to operate within an Industry 4.0 paradigm, i.e., it fulfills typical restrictions of the OT world, while including modern cloud connectivity and administration tools typical of the IT world. The IoT-PLC is designed to interact with devices over different protocols, to extract the relevant information for further cloud consolidation, and to give a standard data format for transparent integration to the cloud.

To take advantage of these capabilities, the IoT-PLC is conceived as a node in the fog layer of an ICPS, in this way it acts as (i) a gateway for field devices that communicate using a variety of sensor and actuator-oriented protocols, particularly wireless; and (ii) a first level controller, manipulating the actuators based on the information it acquires from the sensors and commands it receives from the cloud.

The IoT-PLC is internally structured according to Fig. 1, where each block represents a functional script or process running inside a container that works independently and exchanges information with other blocks based on a publisher/subscriber communication scheme. The blocks at the bottom of Fig. 1 are meant to provide connectivity with field devices and allow the data level to be raised so that higher blocks can work transparently, independent of the particular nature of field devices. Upper blocks in Fig. 1 work with abstracted data and offer functionalities related to the control algorithm and connectivity with: human users, other IoT-PLCs and the cloud.

By running each block in a container it is possible to manage them directly from the cloud. This gives the IoT-PLC flexibility for changing on-the-fly the configuration of the controller without stopping the control process, only the controller block needs to be reloaded. In addition, containerization allows assigning resources to each block depending on the role they have in the control system, increasing reliability and predictability.

An important aspect of ICPSs is closed-loop reconfiguration [20]. For transparent reconfiguration involving a second IoT-PLC, a stateful live migration functionality is included. Live migration is the process

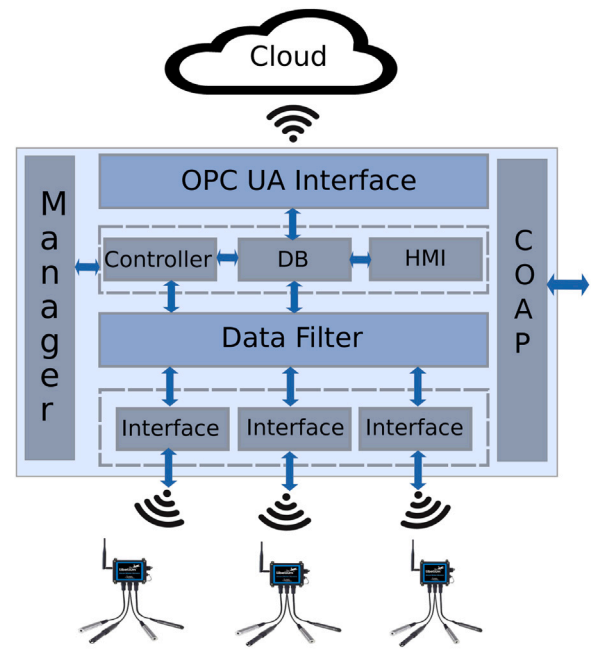


Fig. 1. Internal structure of the IoT-PLC.

when the in-memory state of a running guest OS is recorded, transferred, and recreated at destination. The in-memory state includes the applications, system processes, and resources currently loaded in memory for quick access. It contains the progress (state) of running applications, including data that is currently being used [21]. Live migration makes it possible to restore the progress of the containers in another PLC exactly where the original PLC was, enabling loop reconfiguration with minor impact on control performance.

The building blocks of the IoT-PLC are explained in the following.

3.1. OPC-UA connectivity

Connectivity to the cloud is available using the OPC-UA protocol. OPC-UA is a platform-independent standard over which different systems and devices communicate. OPC-UA can be used in all industrial levels, such as field (sensors, actuators), control systems, Manufacturing Execution Systems (MES) and Enterprise Resource Planning (ERP).

Through OPC-UA, the data collected is sent to the servers in the cloud. In addition, descriptions of devices, controllers and loops can be sent from the cloud to be added or modified during the control process.

Using the OPC-UA block it is also possible to send the necessary commands from the cloud to an IoT-PLC to proceed with the migration of the containers to another device.

OPC-UA connectivity allows the IoT-PLC to integrate transparently with a broad family of control systems. The OPC-UA system architecture models clients and servers as interacting partners. Each system may contain multiple clients and servers. Each client may interact concurrently with one or more servers, and vice versa. OPC-UA defines a common infrastructure model, which specifies: the information model to represent structure, behavior and semantics; the message model to interact between applications; the communication model for data transfer between end-points; and the conformance model to guarantee interoperability.

Using OPC-UA, it is possible to map the functions exposed by the IoT-PLC in a standardized way. Controllers, assigned devices, their configurations and mechanisms to access inputs and outputs can be standardized using the OPC-UA Information Model for IEC 61131-3. The device descriptions and the IoT-PLC itself, together with information related to the personnel involved and the types of processes they

Table 2
Main characteristics of previous efforts and the proposed IoT-PLC.

	Low latency	Control oriented	Horizontal connectivity	Device description	Live block re-configuration	Cloud-based
[19]	✓	✓			✓	
[18]	✓	✓	✓			
[13]	✓	✓				
[17]	✓		✓		✓	
[14]	✓			✓		
[12]		✓				✓
[15]		✓				✓
[16]	✓	✓	✓			
IoT-PLC	✓	✓	✓	✓	✓	

interact with, can be transparently inserted into an ISA-95 compatible system to exchange information with MOM systems. Finally, the IoT-PLC and its description can be included in an AutomationML model of a plant and communication system, besides including a description of the implemented controllers.

3.2. Interfaces with field devices

The IoT-PLC bases its connectivity with field devices on bi-directional interface blocks that translate data from high level IoT-PLC blocks to field devices and vice versa. Each interface block is capable of communicating using only one protocol, e.g., Zigbee, Bluetooth, Wireless Hart, or Wi-Fi. Interface blocks communicate with the upper blocks, Manager and Filter blocks, by means of publication/subscription event messages. By doing so, upper blocks can obtain data whenever it is required, without depending directly on the transmission rate of the field devices or the others building blocks of the IoT-PLC.

Interface blocks are designed in such a way that any upper block can subscribe to the interfaces, so that data can be transferred to or from new functional blocks by simply subscribing to the interfaces. Hence, interface blocks provide a standardized mechanism for interacting with field devices, agnostic to the communication protocol and other technological particularities.

For development, to port a new communication protocol to the IoT-PLC, in addition to the physical hardware, only a containerized interface block must be generated. For updating, a new containerized interface block is sent from the cloud using OPC-UA and loaded in the IoT-PLC.

3.3. CoAP interface

The CoAP interface block is intended to provide connectivity with other IoT-PLCs or fog devices. Using this interface, it is possible to exchange data of devices and controllers, and to give control commands to other IoT-PLCs, such as a control loop termination signal. This is achieved through the use of the CoAP Multicast server discovery feature, resulting in a decentralized discovery process among the devices in a network.

The CoAP protocol, a mature IoT-oriented solution, was chosen because of its proven properties in gateway devices, such as low overhead, reduced latency and decentralized data exchange [22]. These features ensure that communication with other IoT-PLCs will not downgrade performance of the local control processes.

3.4. Data filter

The data filter block operates as an abstraction layer in between the interface blocks and the upper elements. The data filter block gathers the information from the interfaces and preprocesses it for its later use by the upper blocks. The incoming data stream from the interface blocks is preprocessed to reduce noise, repair missing data, and mask outliers by using filters. Filters applied in this block are instanced according to the configuration sent from the cloud, so different filters can be applied depending on the needs of sensors and processes. Validated data is then published to the upper blocks that require the data.

3.5. Database

The database block acts as a consolidation unit. The information is transformed to make it more meaningful when is sent to the cloud or is taken by the HMI. The data consolidated in this block corresponds to the filtered information from sensors, data generated by the control algorithm inside the IoT-PLC, and data sent from the HMI block.

The database block is in charge of sending data to the OPC-UA block to be transmitted to the cloud. It also receives, from the OPC-UA block, the descriptions and configurations of the devices the IoT-PLC will be connected to, and saves them. The database block provides an adaptation layer that makes data independent of the time-scales at which field devices operate.

3.6. Device and controller description

For operation, devices and controllers are represented as virtual devices (a programming object). These virtual devices are intended to be used as an abstraction method to work with real entities, so IoT-PLC applications can interact with data from a high level perspective. To save and manage configurations and properties of the virtual devices the *json* format is used, which gives flexibility to users and developers to store complex data types without facing the restrictions of relational databases.

The virtual devices are based in the resource object model presented in [23], where the model aims to reflect the integration of the physical world and the information space for IoT objects. The resource model used for the IoT-PLC is shown in Fig. 2. In this model every IoT-PLC has a folder object in the OPC-UA server with its controllers assigned, within that folder every controller includes the properties and variables, and the virtual devices representing sensors and actuators (field devices).

For field devices there are two type of descriptions, one is used internally by the IoT-PLC and the other is used by the cloud through OPC-UA. The first type is the device resource with three cyan squares in Fig. 2, whose meaning is as follows: “attributes” is a value that does not normally change (such as manufacturer or specification), “status” refers to the value of some parameters at a certain time and “control” refers to the access control interface information provided by the device. The second type of description includes the three cyan squares and adds the green square, where “history” refers to the operation or data uploaded by the device in the past.

3.7. Manager

The manager block is responsible for taking data from the descriptions of devices and controllers in the database to instantiate virtual devices and controllers, which have the ability to group together to form a control loop. Virtual devices, controller and control loops are represented and used as a programming object in the manager block.

When a control loop is created, the manager starts a process inside a container dedicated to the control task, to which the virtual devices and the control loop are transferred, so the loop object can operate and send

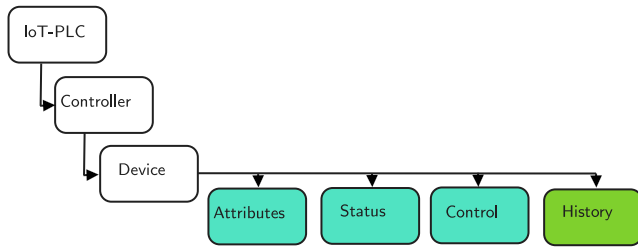


Fig. 2. Model of a virtual device.

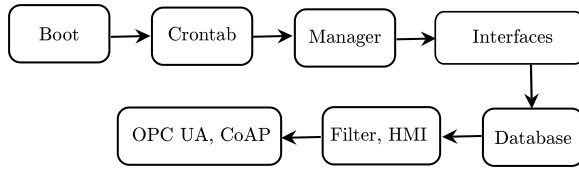


Fig. 3. Booting process of the IoT-PLC.

the generated data to the database block independently of the manager. In this way, the manager can modify the control loop at any time if one or more parameters are required to change, or it can stop the control process, without interfering with other control loops or IoT-PLC blocks.

3.8. Human-machine interface

A human-machine interface (HMI) is added to the IoT-PLC to give a local or external user of the system the option to configure all the parameters involved in the control algorithm, and to monitor the status of the controlled and manipulated variables from anywhere provided the appropriate authorization is given.

The HMI only has one data flow, with the database block, so that in case of modifications from that block the manager and the control processes can interact. When the HMI requires data, it issues a request to the database block, then the database retrieves the information, which is finally presented to the user by the HMI.

3.9. Boot and recovery system

The boot process of the IoT-PLC starts with the cloud giving the signal to a script in a device to start the containers that contain the processes of each block of the IoT-PLC, in case they do not have them, they can be build locally, be obtained from a nearby IoT-PLC or directly from the cloud.

The order of the containers when booting, follows the one shown in Fig. 3, then the same script is called periodically to check if every block is running, in the case a block has been stopped it is immediately restarted to keep the operation as clean as possible.

3.10. Operation flow of the IoT-PLC

The operation of the IoT-PLC follows the sequence diagram shown in Fig. 4. The control process begins when the cloud sends the configuration of the field devices to be used. Then, the IoT-PLC will start looking for the devices, if they are available it receives the availability indicator in the form of a broadcast message. Then the IoT-PLC indicates the field device that it has been assigned the status of available. The next step is to instantiate a virtual device in the controller and establish the connection.

The next part of the process starts when, from the cloud or locally, configuration parameters of a controller are added and, optionally, the cloud subscribes events. Based on these parameters, the IoT-PLC

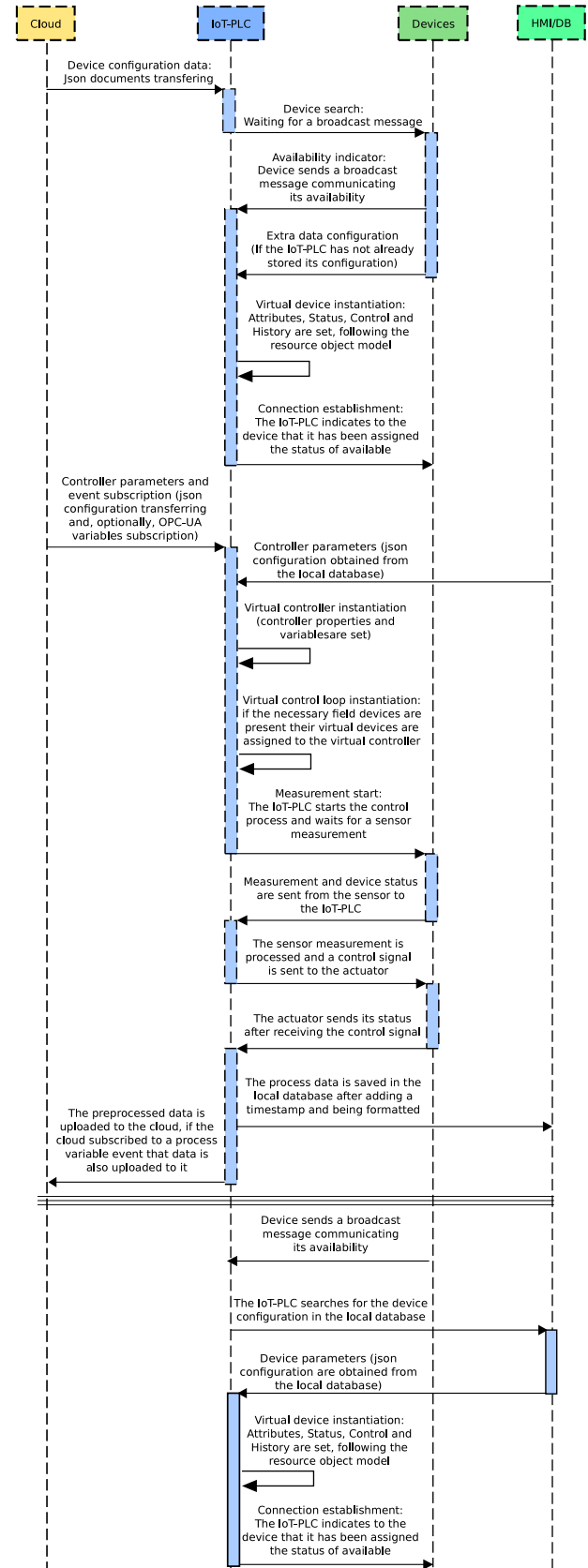


Fig. 4. Sequence diagram of the IoT-PLC.

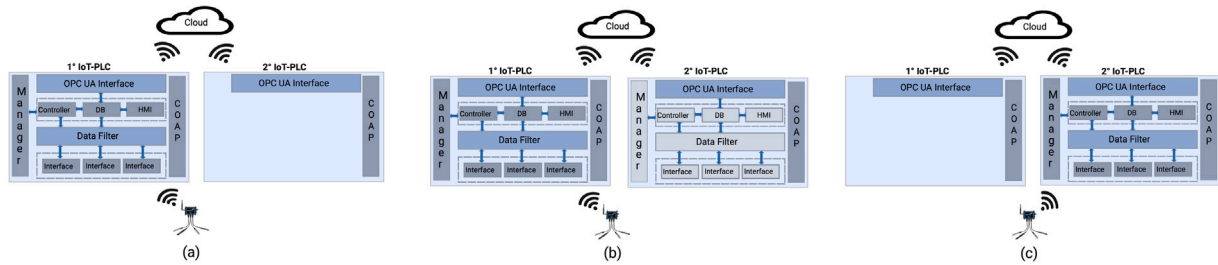


Fig. 5. Container migration process between IoT-PLCs: (a) initial state, (b) intermediate state, containers in the second IoT-PLC uninitiated, (c) final state.

instantiates a virtual controller and, if the necessary sensors and actuators are present, a virtual control loop is instantiated. Then, the data streaming from the sensors is initiated and, with this, the controller starts streaming control signals to the actuators. Process data is also sent to the database and the cloud.

A second scenario for the control process is when the device configuration data is already stored in the IoT-PLC but a device is not present, the IoT-PLC waits until the field device becomes available and then starts the instantiation of its virtual entity.

3.11. Container migration between IoT-PLCs

The design of the IoT-PLC allows the migration of containers between two devices using a stateful live migration process. To this end, the cloud is in charge of orchestrating the migration process, which involves two devices: one with the containers that run the control process, with their assigned devices, and another without the containers. Fig. 5(a) shows the initial state of the migration process.

The operation flow for migration starts by selecting an idle IoT-PLC as migration target, which must have available the required resources for deploying all the containers. Then, the cloud sends instructions to the first IoT-PLC device through OPC-UA, initiating the migration process. The first IoT-PLC is in charge of (i) verifying via CoAP discovery that the second IoT-PLC is available; and (ii) generating a repository with the original containers, without the in-memory state. Then, the cloud orders the second IoT-PLC to get the containers from the first one, when this task is successfully completed the cloud is informed. Fig. 5(b) shows the status of this intermediate stage.

Next, from the cloud, to save the progress of the containers since they were started, the in-memory state process is initiated, this is called container checkpointing. Then, these states are transferred to the second IoT-PLC and started immediately. During the checkpoint process each container is paused to get the in-memory state and after that it is resumed to keep the system up as long as possible.

Finally, the cloud orders the first IoT-PLC to communicate to its assigned devices to migrate the data streaming process to the interfaces of the second IoT-PLC. Finally, the containers are stopped at the first IoT-PLC. From this time on, the control process continues at the second IoT-PLC. The final status of the process is shown in Fig. 5(c).

An HTTP API is used to interact with the generated repository, which allows the use of the GET, HEAD, POST, PUT and DELETE request methods to obtain the necessary containers. The container repository was designed to be created at the IoT-PLC, because in this way the migration of containers does not depend on the quality of the connection between the cloud and the IoT-PLC.

4. Prototype

4.1. Hardware and software

The prototype of the IoT-PLC was implemented in a Raspberry Pi 4B board. In terms of connectivity, the on board Ethernet interface was used to connect the prototype to the Internet and transfer information

to a desktop computer simulating a cloud environment, while for connecting with field devices (sensors and actuators) the on board Bluetooth interface was used together with an XBee S2D ZigBee module (connected via the UART peripheral) that provides a Zigbee (IEEE 802.15.4) capable interface. Data exchange with the serial interface (to connect to the XBee S2D ZigBee module) was achieved using a Python script that uses the pySerial module to access the serial port of the device.

The interface blocks of Fig. 1 communicate with the manager and the filter using the Redis publish and subscribe function, which was chosen because of its speed and stability. The communication between the other blocks was also implemented using Redis in its pub/sub form. The database block was implemented using SQLite, chosen because of its speed and low computational load.

The HMI was developed with the micro web framework Flask and the interactive visualization library Bokeh, both mainly written in Python. The web interface is accessible through any modern browser, as it uses the HTTP protocol.

The OPC-UA interface block was developed using the python-opcua library, in the evaluation the server was located in the desktop computer simulating the cloud and the IoT-PLCs were connected as clients.

The software platform used for containers was Docker, which stands out for being lightweight and allowing a flexible and personalized use of computer resources. The Docker Registry 2.0 implementation for storing and distributing Docker images was used to store the containers.

For container migration, the process was automated using OPC-UA and the CRIU software, which makes possible to freeze a running container and checkpoint it to persistent storage as a collection of files. To evaluate the control process (see next section) with container migration, two prototypes of the IoT-PLC were used over the same local network.

4.2. Evaluation

To evaluate the performance of the IoT-PLC, a simulated four tanks system was used, which is shown in Fig. 6. The four tanks system is a classical control example, whose formulation was taken from [24] and simulated within a Python environment. The four tanks system was implemented in a Beaglebone Black Wireless (BBBW) board, where an actuator data stream is generated using the on-board Bluetooth interface, while a sensor data stream was setup using an XBee S2D ZigBee module attached to the BBBW through the UART peripheral. Fig. 7 presents part of the prototype and the tanks system used in the evaluation.

An IoT-PLC was configured to generate a simple control loop over two interfaces, namely, a Zigbee interface where it receives the data stream from the field sensors, and a Bluetooth interface over which the IoT-PLC sends control commands to the field actuators. In term of controllers, a model-predictive-controller (MPC) and a proportional-integral (PI) controller were instanced, in different evaluation scenarios, to control the water level of the two lower tanks by manipulating the voltage of the pumps. Both controllers compute the output signals as fast as possible when a measurement is obtained from the sensors.

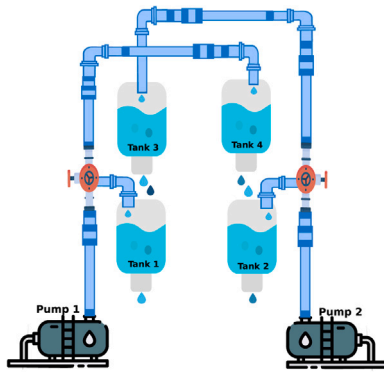


Fig. 6. Four tanks system used in the evaluation.

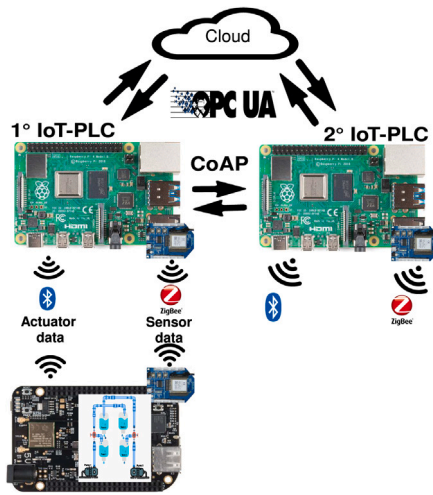


Fig. 7. Part of the IoT-PLC prototype implemented in the laboratory.

A simple low-pass filter was used in the filter block to remove noise from the sensor data stream. The model used by the MPC to control the process was a linearized formulation of the four tank system, which was obtained from [24], the sampling time of the system was 250 ms.

For performance evaluation, a set-point tracking experiment was designed and repeated 20 times. For each realization of the experiment, the initial set-points were set at 12.4 cm and 12.7 cm, for the first and second tank, respectively, and during the experiment the set-points were changed to:

1. Tank 1: 35 cm, Tank 2: 20 cm
2. Tank 1: 25 cm, Tank 2: 39 cm
3. Tank 1: 32 cm, Tank 2: 28 cm

A key functionality of the IoT-PLC is controller reconfiguration at runtime. To illustrate this functionality, a controller switch during operation was performed by starting a set-point tracking experiment using a PID controller and switching to an MPC controller halfway the second and third reference change. This process starts by loading an MPC controller from the cloud and sending configuration data through OPC-UA so that the IoT-PLC can instance a virtual control loop and send it to the MPC controller. When the MPC controller starts commanding the control loop, it tells the container of the PID controller to stop. Fig. 8 presents the results, it can be seen that near second 400 there is a variation in water level produced by the controller switching, after this point, performance improves due to the presence of the MPC controller.

To evaluate the impact of container failures on control performance, an experiment was conducted where three containers were abruptly stopped, one every 30 s. Faults were introduced in the filter, controller

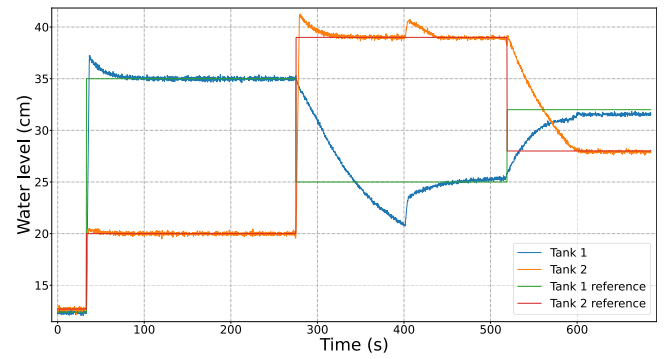


Fig. 8. Set-point tracking results for the controller switching scenario.

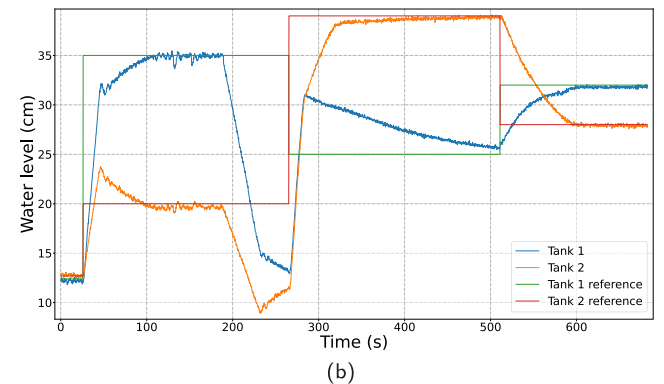
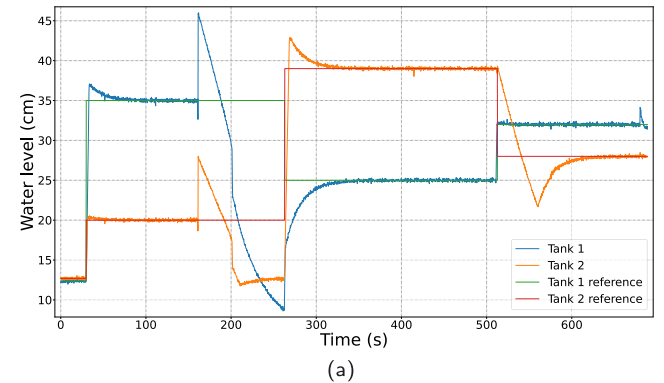


Fig. 9. Set-point tracking results for the container failure scenario. (a): Operation with a PID controller; (b): Operation with MPC controller.

and HMI, in that order. Fig. 9 presents the results for a set-point tracking experiment, it can be seen that container failures deteriorate performance yet the system recovers quickly as soon as the containers are restarted by the manager, without the need of a full reboot of the device. Containerization and the use of pub/sub communication contributes to avoiding a cascading failure, limiting the impact to a single functional block of the IoT-PLC.

In order to evaluate container migration, a second Raspberry Pi 4B board was setup to receive them. Live migration started two minutes after the first set-point change. While the migration process is running, the second set-point change was applied, this was timed to happen after the container repository was ready. The last set-point change occurred just after the second IoT-PLC started to control the system.

During live migration, the containers that did not require an in-memory state snap were OPC-UA interface, CoAP interface and Database. The interface blocks were already initiated at the second IoT-PLC before the migration is started, and the Database block does not

Table 3

Container size and average checkpoint duration. A dash means that the container was not paused.

Container block	Size (kilobytes)	Checkpoint duration (seconds)	
		PID	MPC
Bluetooth	5.4	3.3	3.9
Filter	6.1	5.0	7.4
HMI	4,949	53.9	54.1
Manager	16.9	17.7	15.7
Xbee	8	3.9	5.4
PID controller	11	9.7	–
MPC controller	2,36,206	–	7.7
CoAP	6.1	–	–
Database	5,04,493	–	–
OPCUA	15.6	–	–
Python base	554,711	–	–

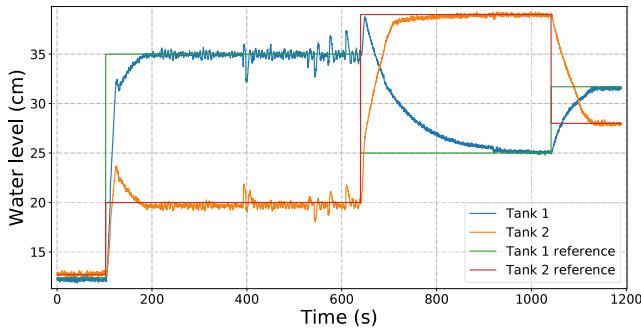


Fig. 10. Set-point tracking results of the wireless loop over two interfaces when using an MPC controller.

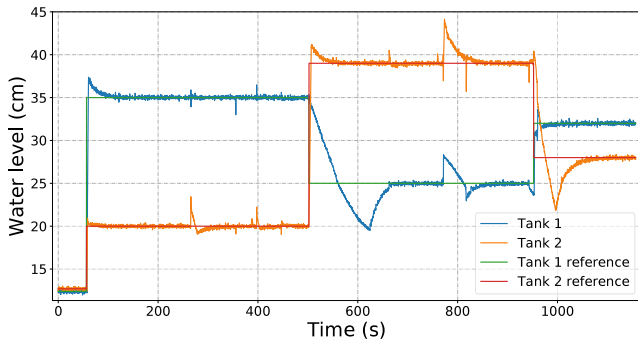


Fig. 11. Set-point tracking results of the wireless loop over two interfaces when using a PI controller.

need to save its states in memory since its operation does not change if it starts from scratch or continues from the point where it was paused before being migrated.

The size of the container layers is shown in Table 3, the base layer, which has the libraries required for every container application, is called “Python base” and the layers containing the applications are added over that layer.

Set-point tracking results of a particular realization are presented in Figs. 10 and 11 for each of the two evaluated controllers: MPC and PI. It can be seen that the IoT-PLC is able to generate a wireless control loop over two interfaces, and keep the instanced controllers running and operating within the latency required by the tank system to remain stable, while it is filtering, saving, showing the data of the entire control process in the HMI, and running the live migration to the second IoT-PLC.

It should be noted that while saving the in-memory state, during the MPC test a small drop in the water level of the second tank

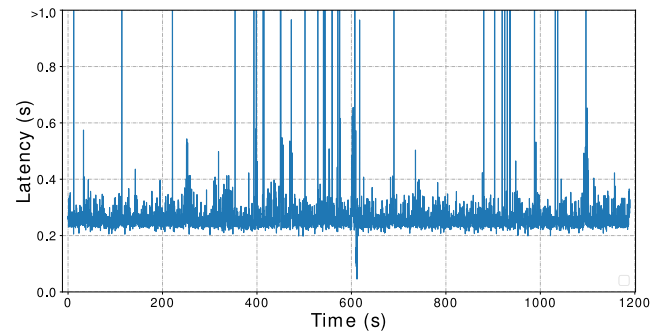


Fig. 12. Measured loop latency when using an MPC controller.

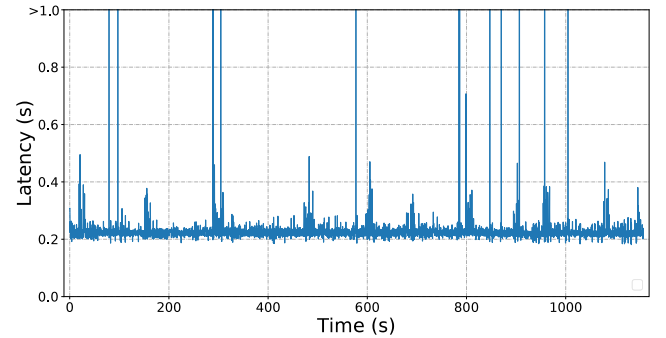


Fig. 13. Measured loop latency when using a PI controller.

can be seen, while during the PI controller test a big spike occurred. These phenomena took place during the period in which the controller block, filter and wireless interfaces were paused. Since the mentioned containers are necessary for the IoT-PLC to send a control signal to the actuator, when paused the control process is also momentarily paused. The average time each container is paused is shown in Table 3.

During testing, packet losses in the ZigBee link were observed. For the experiment shown in Fig. 10, the corresponding loop latency (time elapsed between the sensor sending a measurement and the actuator receiving a signal) measured is shown in Fig. 12, where a value over 1 s implies a packet loss. By analyzing both figures, is evident that, as it is well known, the large latency in the sensor-controller link negatively affects the performance in the controller. As already mentioned, performance deterioration after the second reference change is due to the pause of the containers assigned to the controller block, filter and wireless interfaces. A similar situation was observed during the use of the PI controller, yet packet losses presented less intensity. Fig. 13 shows the measured loop latency for the experiment shown in Fig. 11.

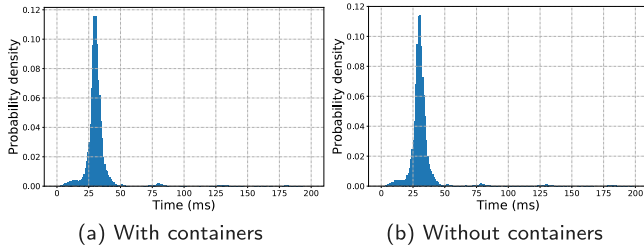
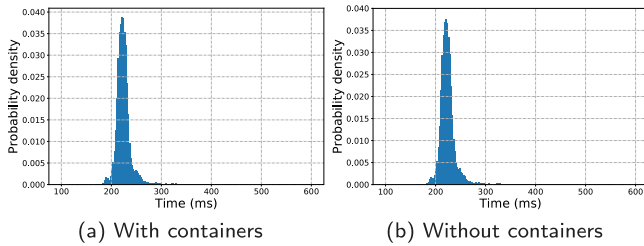
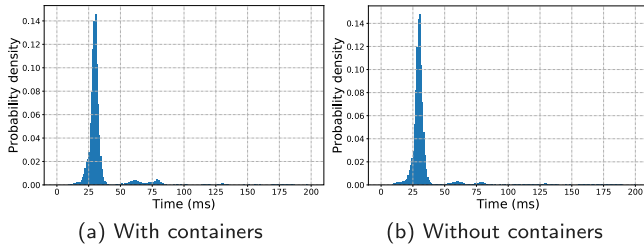
To determine the effect of container usage in the operation of the IoT-PLC, a new test was performed, which includes the same reference changes but without container migration. To conduct the evaluation, a new prototype was built using the same hardware but without the use of containers, hence similar to the device proposed in [16]. We refer to this device as the container-free prototype. To evaluate the impact of containers, the control latency (time elapsed since the moment the IoT-PLC receives a measurement until it sends a control signal), and the loop latency were measured.

Figs. 14, 15, 16 and 17 present the results obtained for 20 realizations per controller yielding 3500 datapoints for the PI case and 3100 datapoints for the MPC. Results are summarized in Table 4, a minor, yet not significant, improvement can be observed when containers are used both in terms of mean and dispersion. The use of containers adds complexity, yet it provides a flexible mechanism for implementing the functionalities of the PLC and it increases modularity and fault

Table 4

Loop latency and control latency for both controllers under operation with and without containers. All values are given in milliseconds.

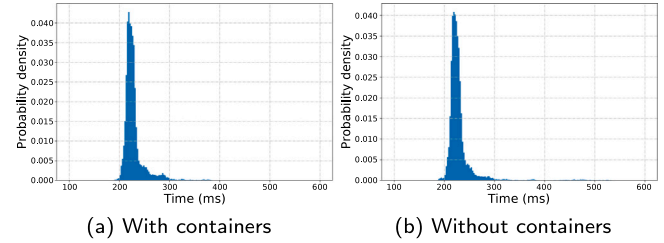
	PI controller		MPC controller	
	Average	Standard deviation	Average	Standard deviation
Loop latency (without containers)	230.8	58.7	233.5	65.7
Loop latency (with containers)	227.9	55.3	232.2	65.4
Control latency (without containers)	34.7	37.8	35.7	41.9
Control latency (with containers)	32.6	35.6	35.0	41.8

**Fig. 14.** Histograms of the control latency: time used by the IoT-PLC to process the received data, when using a PI controller.**Fig. 15.** Histograms of the loop latency: total time between the sensor sending a measurement and the actuator receiving a signal, when using a PI controller.**Fig. 16.** Histograms of the control latency: time used by the IoT-PLC to process the received data, when using an MPC controller.

tolerance at no cost in terms of latency, in fact, although of no statistical significance, it mildly reduces latency.

Although histograms in Figs. 14, 15, 16 and 17 do not show a conclusive result about the effect of the use of the containers on the total time, they do show how the use of the Zigbee and Bluetooth interfaces produce a high variability in the latency. This is consistent with the latency graph of Figs. 12 and 13. Resorting to a design strategy as the one presented in [25], where wireless radios are directly commanded by the main microprocessor could help in reducing the extra latency produced by the wireless interfaces.

In order to illustrate the increased reliability of the containerized IoT-PLC with respect to the container-free prototype, a faulty behavior in the OPC-UA and CoAP blocks was introduced, which causes these blocks to linearly increase their RAM usage. This situation resembles an external attack via those communication interfaces. The faulty scenario was evaluated in the containerized IoT-PLC and the container-free

**Fig. 17.** Histograms of the loop latency: total time between the sensor sending a measurement and the actuator receiving a signal, when using an MPC controller.**Table 5**

Latency obtained when closing the loop over CoAP and OPC-UA interfaces. All values are given in milliseconds.

	PI controller		MPC controller	
	Average	Standard deviation	Average	Standard deviation
CoAP loop latency	273.9	71.1	289.4	94.9
OPC-UA loop latency	595.2	277.5	637.1	328.8
CoAP control latency	50.4	60.2	57.5	57.1
OPC-UA control latency	336.4	213.2	381.2	193.1

prototype. Tests consisted in triggering the faulty behavior twice during a set-point tracking experiment, particularly during the first and second reference change. The values of water levels and total RAM usage were recorded and are presented in Fig. 18 for a PID and an MPC controller. It can be seen that the container-free device cannot reach the second reference change since faulty processes occupied all the RAM, and start using SWAP memory, overloading the device and finally producing a crash that had to be solved via a reboot. The containerized IoT-PLC, on the other hand, experiences an increased RAM usage that is limited by the resources allocated to the faulty containers. When the manager detects a faulty (non-responsive) behavior at the OPC-UA and CoAP containers, it kills and restarts the processes, recovering the nominal behavior and effectively controlling the tanks system.

To illustrate the advantages of horizontal connectivity using the CoAP interface, a setup was configured where the IoT-PLC does not access the actuator directly using the field interfaces, but rather it needs to reach the actuator via an intermediate node. Tests were conducted closing the loop over the CoAP interface, and also over the OPC-UA interface, i.e., using the cloud to reach the actuator, to have a baseline for comparison. To keep the experiment as fair as possible, both the cloud and a second IoT-PLC acting as intermediate node were placed at one hop in a local area network (LAN) operating using WiFi (IEEE 802.11n). Table 5 presents the results in terms of latency for both scenarios, it can be seen that closing the loop over OPC-UA dramatically increases both loop and control latency (cf. Table 4), while CoAP represents only a mild increase. This is due to the lightweight nature of CoAP that results in a much lower overhead compared to OPC-UA. Fig. 19 presents the results of a set-point tracking experiment, it can be seen that the latency introduced when closing the loop over OPC-UA deteriorates performance, while when closing the loop over CoAP performance is not affected (cf. Figs. 10 and 11).

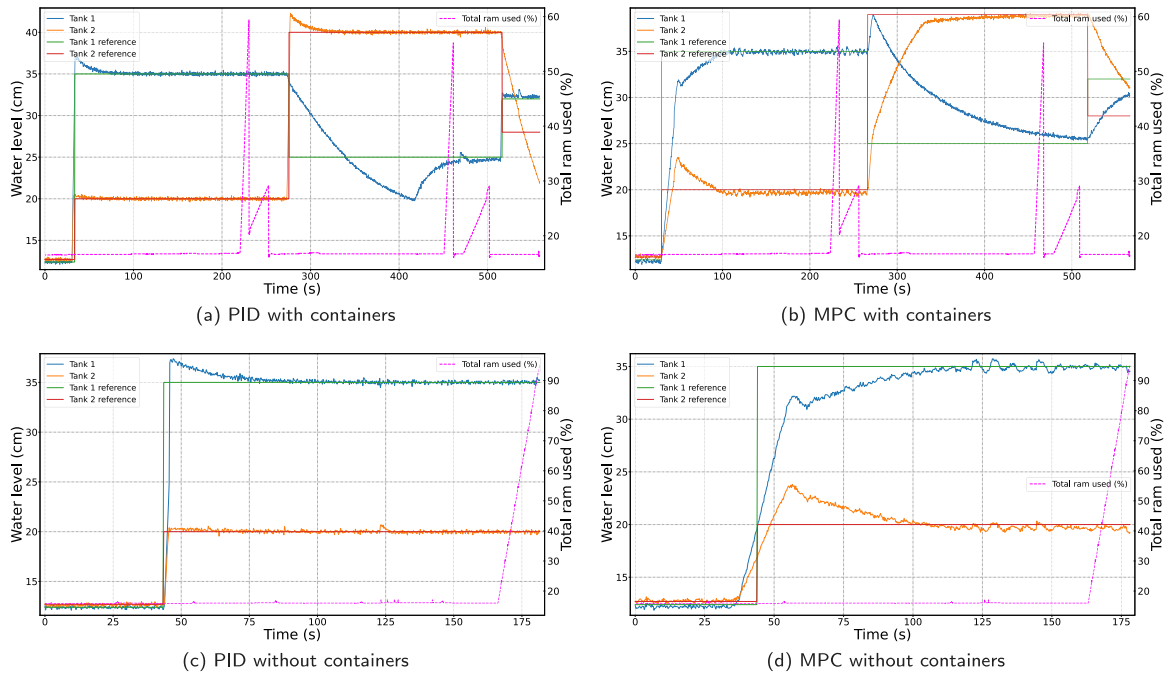


Fig. 18. Set-point tracking results and RAM usage for the faulty PLC tests. (a): Operation with a PID controller and containers; (b): Operation with MPC controller and containers; (c): Operation with PID controller and without containers; (d): Operation with MPC controller and without containers.

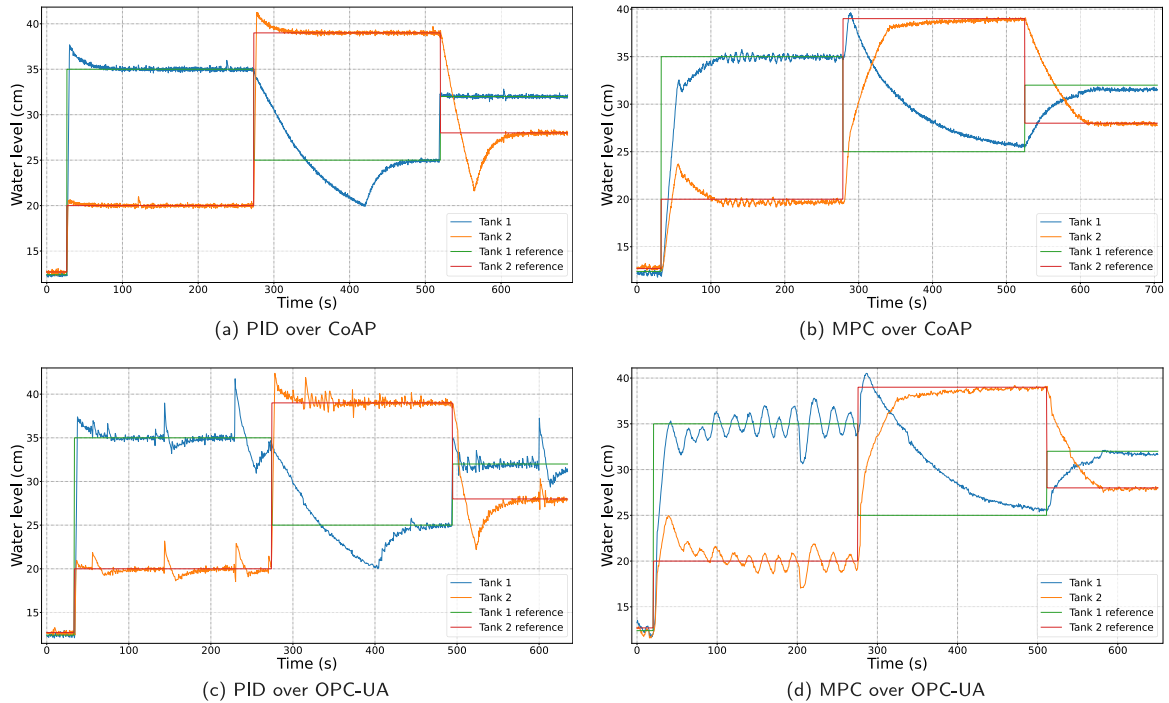


Fig. 19. Set-point tracking results when closing the loop over high-level interfaces. (a): Operation with a PID controller over a CoAP interface; (b): Operation with MPC controller over a CoAP interface; (c): Operation with a PID controller over an OPC-UA interface; (d): Operation with MPC controller over an OPC-UA interface.

4.3. Discussion

The experimental evaluation show that the proposed IoT-PLC is able to command a control loop with good performance in different scenarios. The IoT-PLC enables using a three-tier architecture (field, fog and cloud) for a control system, where the IoT-PLC plays the role of a fog node [26], processing data and taking actions in real time,

while at the same time is consolidating information to be sent to the cloud for analysis and decision making at a different time-scale. In this regard, the set-point tracking results when closing the loop over the CoAP interface are significant, since it validates the idea of having a low-overhead intermediate network of fog devices over which control loops can operate without compromising performance, in contrast to

treating each PLC as an isolated unit and use the cloud to close the loop.

In its current form, the main disadvantages of the proposed device are: (i) the relatively high latency of wireless interfaces, and (ii) the absence of a runtime environment for control applications fully compliant with IEC 61131-3. The former renders the device suitable only for process control and building automation (see Table 1) and not appropriate for faster systems as power electronics control, while the latter precludes the use of existing control logic directly in the IoT-PLC and implies an extra engineering effort to develop a customized control logic for the IoT-PLC. To overcome these disadvantages, a future version of the device should: (i) explore the use of new wireless technologies, particularly 5G; and (ii) include a containerized IEC 61131-1 engine, following the directions given in [9].

5. Conclusion

An IoT-PLC, a programmable logical controller (PLC) designed for the Industry 4.0, is presented, prototyped and evaluated in this work. The IoT-PLC is designed as a fog layer node, and implements a variety of communication interfaces: (i) a series of interfaces to communicate with field sensors and actuators; (ii) a CoAP interface to communicate with neighboring IoT-PLCs; and (iii) an OPC-UA interface to communicate with a cloud. Moreover, it includes a local database and a human-machine-interface for configuration and data visualization.

The modular structure based on containers allows the IoT PLC to safely isolate the different processes that run in it. Moreover, containerization stimulates moving from the classic hardware-based PLC to one that has virtual functions of control, processing and visualization of data and it can act as a bridge between industrial automation applications and IT technologies.

Evaluation in controlling a four tanks system, shows that the IoT-PLC prototype is able to generate and command a wireless control loop over two interfaces, with an MPC and a PI controller instanced in the loop. Set-point tracking results indicate that the latency achieved is adequate for process control applications in which the control process variables do not change fast, this comes from the fact that the latency resulting from the inclusion of wireless interfaces is in the hundreds of milliseconds range. Evaluation results also show that the live migration process during the control of the four-tank system was successful, although the pauses each container needs for migrating interfere with set-point tracking and produce a minor performance deterioration.

Future research includes making the IoT-PLC fully IEC 61131 compliant and conducting extensive testing in a real-scale processing plant. Future research on the topic also includes security, as IT/OT integration poses threads to the physical process under control.

CRedit authorship contribution statement

Jacob Mellado: Software, Formal analysis, Investigation, Writing. **Felipe Núñez:** Conceptualization, Methodology, Formal analysis, Writing, Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] Y. Lu, Industry 4.0: A survey on technologies, applications and open research issues, *J. Ind. Inf. Integr.* 6 (2017) 1–10.
- [2] A.W. Colombo, S. Karnouskos, O. Kaynak, Y. Shi, S. Yin, Industrial cyberphysical systems: A backbone of the fourth industrial revolution, *IEEE Ind. Electron. Mag.* 11 (1) (2017) 6–16.
- [3] R. Harrison, D. Vera, B. Ahmad, Engineering methods and tools for cyber-physical automation systems, *Proc. IEEE* 104 (5) (2016) 973–985.
- [4] F. Núñez, S. Langarica, P. Díaz, M. Torres, J.C. Salas, Neural network-based model predictive control of a paste thickener over an industrial internet platform, *IEEE Trans. Ind. Inf.* 16 (4) (2020) 2859–2867.
- [5] S. Langarica, C. Rüffelmacher, F. Núñez, An industrial internet application for real-time fault diagnosis in industrial motors, *IEEE Trans. Autom. Sci. Eng.* 17 (1) (2020) 284–295.
- [6] A. Gilchrist, *Industry 4.0*, Apress, Berkeley, CA, 2016.
- [7] T. Bangemann, M. Riedl, M. Thron, C. Diedrich, Integration of classical components into industrial cyber-physical systems, *Proc. IEEE* 104 (5) (2016) 947–959.
- [8] B. Galloway, G.P. Hancke, Introduction to industrial control networks, *IEEE Commun. Surv. Tutor.* 15 (2) (2013) 860–880.
- [9] T. Goldschmidt, S. Hauck-Stattelmann, S. Malakuti, S. Grüner, Container-based architecture for flexible industrial control applications, *J. Syst. Archit.* 84 (2018) 28–36.
- [10] Z. Pang, M. Luvisotto, D. Dzung, Wireless high-performance communications: The challenges and opportunities of a new target, *IEEE Ind. Electron. Mag.* 11 (3) (2017) 20–25.
- [11] M. Chiang, T. Zhang, Fog and IoT: An overview of research opportunities, *IEEE Internet Things J.* 3 (6) (2016) 854–864.
- [12] O. Givehchi, J. Imtiaz, H. Trsek, J. Jasperneite, Control-as-a-service from the cloud: A case study for using virtualized PLCs, in: 2014 10th IEEE Workshop on Factory Communication Systems, WFCS 2014, 2014, pp. 1–4.
- [13] T. Cruz, P. Simões, E. Monteiro, Virtualizing programmable logic controllers: Toward a convergent approach, *IEEE Embed. Syst. Lett.* 8 (4) (2016) 69–72.
- [14] B.R. Ferrer, W.M. Mohammed, E. Chen, J.L.M. Lastra, Connecting web-based IoT devices to a cloud-based manufacturing platform, in: IECON 2017 - 43rd Annual Conference of the IEEE Industrial Electronics Society, 2017, pp. 8628–8633.
- [15] T. Goldschmidt, M.K. Murugaiah, C. Sonntag, B. Schlich, S. Biallas, P. Weber, Cloud-based control: A multi-tenant, horizontally scalable soft-PLC, in: 2015 IEEE 8th International Conference on Cloud Computing, 2015, pp. 909–916.
- [16] I.T. Popović, A.Z. Rakić, The fog-based framework for design of real-time control systems in internet of things environment, in: 2018 International Symposium on Industrial Electronics, INDEL, 2018, pp. 1–6.
- [17] K. Dolui, C. Kiraly, Towards multi-container deployment on IoT gateways, in: 2018 IEEE Global Communications Conference, GLOBECOM, 2018, pp. 1–7.
- [18] A. Badar, D.Z. Lou, U. Graf, C. Barth, C. Stich, Intelligent edge control with deterministic-IP based industrial communication in process automation, in: 2015 15th International Conference on Network and Service Management, CNSM, 2019, pp. 1–7.
- [19] M. Azarmipour, H. Elfaham, C. Gries, U. Epple, PLC 4.0: A control system for industry 4.0, in: IECON 2019 - 45th Annual Conference of the IEEE Industrial Electronics Society, vol. 1, 2019, pp. 5513–5518.
- [20] J.J. Mendoza, F. Núñez, Blockchain-driven on-demand control loops over IoT environments, *IEEE Access* 7 (2019) 157524–157534.
- [21] A. Machen, S. Wang, K.K. Leung, B.J. Ko, T. Salonidis, Live service migration in mobile edge clouds, *IEEE Wirel. Commun.* 25 (1) (2018) 140–147.
- [22] P. Bellavista, A. Zanni, Towards better scalability for IoT-cloud interactions via combined exploitation of MQTT and CoAP, in: 2016 IEEE 2nd International Forum on Research and Technologies for Society and Industry Leveraging a better tomorrow, RTSI, 2016, pp. 1–6.
- [23] L. Lan, R. Shi, B. Wang, L. Zhang, An IoT unified access platform for heterogeneous sensing devices based on edge computing, *IEEE Access* 7 (2019) 44199–44211.
- [24] K.H. Johansson, The quadruple-tank process: a multivariable laboratory process with an adjustable zero, *IEEE Trans. Control Syst. Technol.* 8 (3) (2000) 456–465.
- [25] T. Herrera, F. Núñez, Design and prototyping of a thread border router based on a non network-co-processor architecture, *IEEE Access* 8 (2020) 60613–60625.
- [26] A. Yousefpour, C. Fung, T. Nguyen, K. Kadiyala, F. Jalali, A. Niakanlahiji, J. Kong, J.P. Jue, All one needs to know about fog computing and related edge computing paradigms: A complete survey, *J. Syst. Archit.* 98 (2019) 289–330.